Tema 9

Optimización

Algoritmos de enjambre

Índice

Esquema 2

Ideas clave 3

9.1 Introducción y objetivos 3

9.2 Objetivos 4

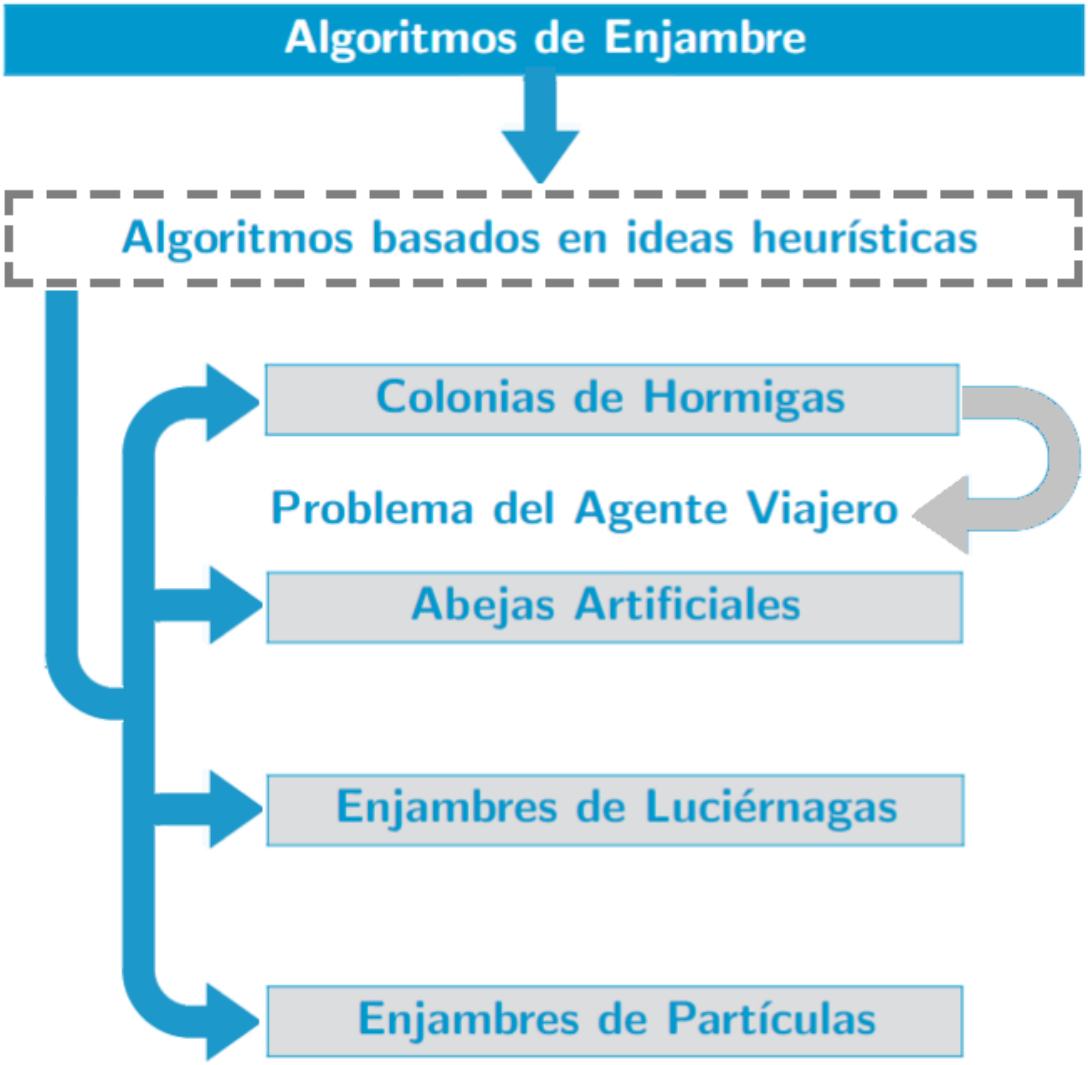
9.3 Algoritmos de colonia de hormigas 4

9.4 Algoritmos de enjambre de partículas 9

9.5 Algoritmos de Abejas Artificiales 19

9.6 Algoritmos de enjambres de luciérnagas 21

Esquema


Algoritmos de Enjambre
Algoritmos basados en ideas heurísticas
Colonias de Hormigas
Problema del Agente Viajero
Abejas Artificiales
Enjambres de Luciérnagas
Enjambres de Partículas

Ideas clave

9.1 Introducción y objetivos

Los algoritmos de enjambre han emergido en las últimas décadas como algoritmos para la resolución y optimización de problemas diversos en distintos tipos de sistemas. El nombre proviene de la idea intuitiva de los distintos enjambres de animales, tradicionalmente insectos, aunque algunos de ellos se inspiran en bandadas de pájaros. El objetivo se basa en reproducir el comportamiento animal en el que una colectividad actúa como un todo para conseguir un beneficio mayor. A menudo, se consideran algoritmos de inteligencia artificial, en los que distintos individuos colaboran para conseguir un determinado objetivo, normalmente optimizar la ruta entre dos puntos, optimizar el consumo de combustible o la superficie ocupada.

Los dos conceptos fundamentales de los algoritmos de enjambre son:

- La **auto-organización**: Entendemos por auto-organización a la capacidad de evolución de los agentes o individuos de un sistema para alcanzar la solución final sin ayuda externa. Según Bonabeau, la auto-organziación se basa en cuatro principios fundamentales: las retroacciones positivas y negativas, las fluctuaciones y las interacciones múltiples. Las retroacciones nos sirven amplificar y estabilizar el problema, las fluctuaciones para incorporar aleatoriedad y las interacciones múltiples para distribuir la información dentro del enjambre.
- La **división de tareas**: definimos la división de tareas como la capacidad de ejecutar de forma simultanea distintas tareas que son realizables por individuos distintos. Así, se divide un problema complejo en distintos más sencillos que son abarcables para cada individuo dentro del sistema.

En el tema anterior hemos visto una clase de algoritmos de enjambre, los algoritmos genéticos. Recordemos que en los algoritmos genéticos comprobamos distintas soluciones posibles y las evaluamos. A continuación dejamos que el sistema evolucione hasta llegar a la solución definitiva. Podemos identificar la auto-organización mediante los pasos de selección, mutación y reproducción de los distintos individuos. La división de tareas se aprecia claramente al tener en cada iterado n individuos explorando su solución y determinando la más adecuada en cada caso.

9.2 Objetivos

El objetivo del tema será determinar el mejor valor para una función de coste $f(x)$.

En este tema nos vamos a centrar en la realización de algoritmos basados en poblaciones de animales. Vamos a trabajar los siguientes 4 tipos de algoritmos:

1. Algoritmos de colonias de hormigas.
2. Algoritmos de enjambres de partículas.
3. Algoritmos de Abejas artificiales.
4. Algoritmos de enjambres de luciérnagas.

9.3 Algoritmos de colonia de hormigas

Los algoritmos de colonias de hormigas (ACO, del inglés Ant Colony Optimization) forman parte de la familia de métodos metaheurísticos inspirados por la tesis doctoral de Marco Dorigo en 1992. Su inspiración se basa en cómo una colonia de hormigas real realiza la detección de alimento y comida. El algoritmo está compuesto por cuatro componentes:

- Las **hormigas**. Son los individuos del sistema. Serán los encargadas de realizar la búsqueda del sistema.
- Las **feromonas**. Cuando una hormiga se desplaza por una superficie, deja un rastro de feromonas que se va evaporando con el tiempo. Cuanto más hormigas recorren un mismo camino, el rastro es más intenso. Si una única hormiga ha pasado por un camino, el rastro se irá degradando hasta llegar a desaparecer, por lo que difícilmente otras hormigas lo vayan a recorrer. A imagen y semejanza de la realidad, las hormigas del sistema dejarán un rastro de feromonas en el espacio de búsqueda que indican la intensidad del camino dejado. Dicha intensidad actúa como elemento de memoria del sistema y permite determinar el mejor sistema posible.
- Las **acciones heurísticas**. Estas acciones permiten recoger la información de todo el sistema, de manera que se consiga información cruzada de todas las hormigas y determine cuándo debemos añadir nueva información. Estas acciones dependerán del sistema que tengamos, en ocasiones no serán necesarias.
- El **control descentralizado**. Dicho control se emplea para mantener el control robusto en la dinámica del sistema. Esto permitirá introducir nuevas hormigas o variaciones en el caso en que alguna hormiga desaparezca o llegue a una solución no factible.

Este tipo de algoritmos es altamente interesante para la resolución de caminos hamiltonianos del estilo del problema del viajante. El algoritmo sigue la lógica siguiente:

1. Inicializamos los caminos de feromonas (lanzamos exploradoras).
2. Mientras no se cumpla el criterio de parada:
   a) Construir una nueva solución componente a componente.
   b) Aplicar las acciones heurísticas del sistema.
   c) Actualizamos los caminos de feromonas.

Para inicializar el sistema, se lanza un conjunto de hormigas o entidades que explorarán el espacio de soluciones. Estas soluciones seguirán las siguientes normas:

- No visitarán un mismo nodo más de una vez.
- La probabilidad de escoger el siguiente nodo será mayor cuanto menor sea la distancia que los separa.
- A mayor intensidad de feromonas en un vértice que une dos nodos, mayor será la probabilidad de seguir por ese nodo.
- Una vez se encuentra una solución factible, se asignan nuevas feromonas en función de la longitud del trayecto.
- Las feromonas se disipan en cada nueva iteración.

Durante la inicialización la condición número 3 no se aplica puesto que todavía no hay feromonas en el sistema.

A partir de la segunda generación, las nuevas hormigas se moverán por el espacio de soluciones siguiendo la siguiente **regla aleatoria de transición proporcional**, la probabilidad de un movimiento viene dada por:

$$p_{ij}^k(t) = \begin{cases} \dfrac{\tau_{ij}(t)^\alpha \eta_{ij}^\beta}{\sum_{l \in \mathcal{J}_i^k} \tau_{il}(t)^\alpha \eta_{il}^\beta} & \text{, si } j \in \mathcal{J}_i^k, \\ 0 & \text{, si } j \notin \mathcal{J}_i^k. \end{cases}$$

Donde $\mathcal{J}_i^k$ es el conjunto de nodos factibles para la k-ésima hormiga partiendo del nodo i. η_{ij} es la visibilidad del nodo j desde el nodo i y se representa como el inverso de la distancia entre los nodos $\eta_{ij} = 1/d_{ij}$ y $\tau_{ij}(t)$ es la intensidad de las feromonas en el vértice que une los nodos i y j en el instante t. Los parámetros α y β son los parámetros de control del sistema, estos parámetros controlan la importancia que le damos a la intensidad de feromonas y a la visibilidad que tiene cada arista. Dependiendo del sistema, deberemos emplear unos valores u otros.

Cuando la hormiga llega a una solución factible se actualizan las feromonas de los vértices por los que ha pasado añdiendo una cantidad $\Delta\tau_{ij}^k$ de feromanas mediante la

fórmula:

$$\tau_{ij}^{k} = \begin{cases} \dfrac{Q}{L^{k}(t)} & \text{, si } (i,j) \in T^{k}(t), \\ 0 & \text{, si } (i,j) \notin T^{k}(t). \end{cases}$$

Donde $T^{k}(t)$ son las aristas por las que ha pasado la hormiga k, $L^{k}(t)$ es la longitud del camino recorrido por la hormiga y Q es un parámetro de control del método.

Cuando se finaliza con todas las hormigas que se han planificado para el tiempo t se realiza la actualización de las feromonas en cada nodo. De forma que se considera:

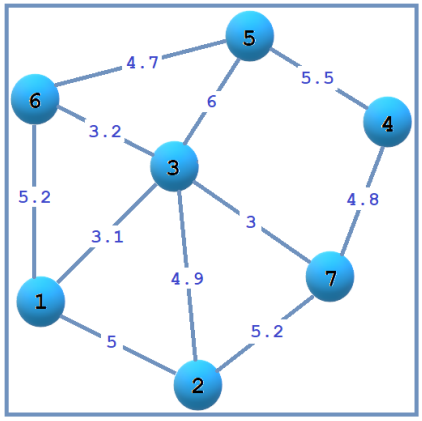
$$\tau_{ij}(t+1) = (1-\rho)\tau_{ij}(t) + \sum_{k=1}^{m} \Delta\tau_{ij}^{k}(t)$$

Donde ρ representa el parámetro de disipación de las feromonas. Si es cercano a 0 las feromonas persisten durante mucho tiempo mientras que si es cercano a 1 las feromonas se disipan rápidamente.

Como podemos ver, el método requiere de cuatro parámetros de ajuste α, β, Q y ρ. Cada problema a tratar requiere de unos valores adecuados. Fijar estos parámetros depende ampliamente de la experiencia que tengamos y de la experimentación con el problema mismo.

Ejemplo 1.

Un agente necesita ir del la ciudad 1 a la ciudad 4 y para ello es necesario encontrar el camino hamiltoniano que optimice el proceso.



El siguiente algoritmo fue presentado por Andrés Herrera Román en 2023 (este algoritmo puede hallarse en MathWorks)

Algoritmo de Colonia de Hormigas - Andrés Herrera Román - 2023

```
clc
clear all
close all

s = [1 1 1 2 2 6 6 3 3 5 7];
t = [2 3 6 7 3 3 5 7 5 4 4];
w=[5 3.1 5.2 5.2 4.9 3.2 4.7 3 6 5.5 4.8];
n=0.1.*ones(1,11);
G = graph(s,t,w);
plot(G,'EdgeLabel',G.Edges.Weight)
p=0.001;
for i=1:100
    %hormiga 1
    [camino,costo] = recorrido(s,t,w,n);
    %hormiga 2
    [camino1,costo1] = recorrido(s,t,w,n);
    n1=zeros(1,11);
    n2=zeros(1,11);
    [delta,qwe] = Feromonas(s,t,camino,w);
    n1(qwe)=delta;
    [delta1,qwe1] = Feromonas(s,t,camino1,w);
    n2(qwe1)=delta1;
    n=(1-p).*n+n1+n2;
end
camino
costo
```

Lo que produce como respuesta: `camino = 1 3 7 4` y `costo = 10.9000`, produciendo el siguiente grafo:

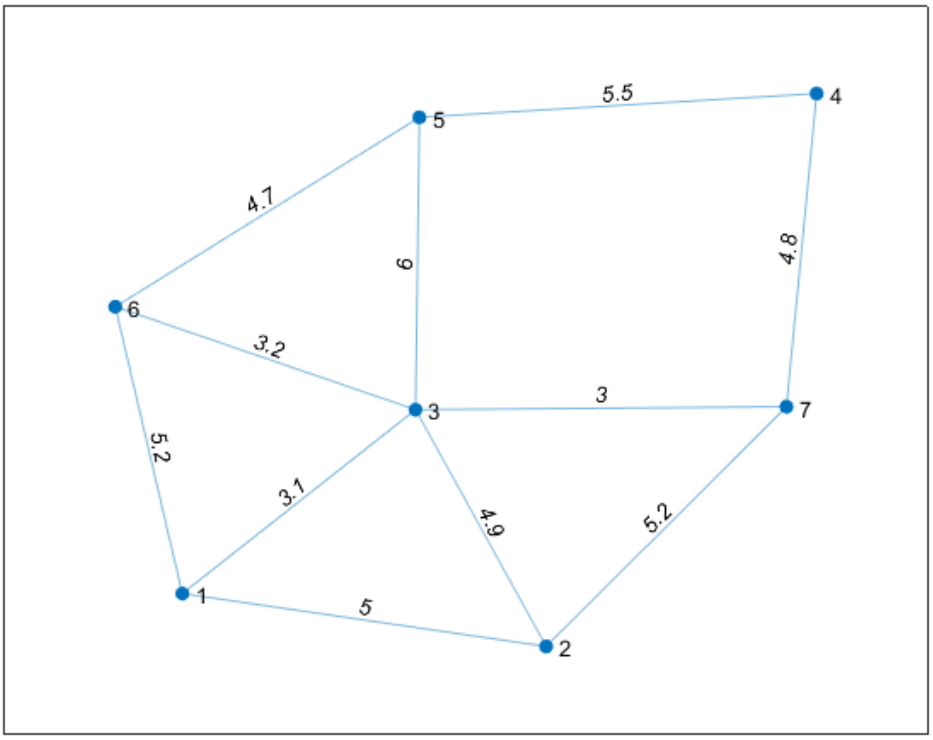


Figura 1: Grafo para el PAV con Algoritmo de Colonia de Hormigas

9.4 Algoritmos de enjambre de partículas

Los **algoritmos de enjambre de partículas** son una técnica de cálculo evolutivo utilizada en optimización. Estos, fueron introducidos por Kennedy y Eberhart (1995), donde la idea intuitiva que guía a la heurística es la emulación de una bandada de pájaros o cardúmenes de peces. En una bandada de pájaros existe un líder que guía a la bandada y el resto de individuos se mueve siguiendo las instrucciones de dicho líder. Además, se establecen diferentes comportamientos alrededor de los diferentes individuos para mejorar la exploración y detección de mínimos locales.

Los individuos interactúan unos con otros, mientras aprenden de sus propia experiencia. Poco a poco, los miembros de la población pasan a mejores regiones del problema

espacio.

En PSO (Particular Swarm Optimization, por sus siglas en inglés), una colección de partículas (llamada “enjambre”) se mueven en el espacio de búsqueda buscando la mejor solución a un problema de optimización.

❶ Su **velocidad** se ve afectada tanto por la **posición** de la partícula con la mejor aptitud (**mejor fitness global**), como por la mejor forma aptitud propia de cada partícula (**mejor fitness local**).

❷ La **aptitud** (**Fitness**) también se refiere a qué tan bien se desempeña una partícula: en una bandada de pájaros, esto podría ser qué tan cerca está un pájaro de una **fuente de alimento**. En un algoritmo de optimización esto se refiere a la **proximidad** de la partícula al global óptimo.

En la heurística se establecen tres tipos de comportamientos:

- La **separación** es el comportamiento por el que se evita saturar un mismo punto que concentre todos los individuos del enjambre. Esto permite realizar una exploración más extensa del sistema.
- La **alineación** permite que todos los individuos se muevan en la dirección promedio. Esto provoca que si hay una dirección favorecida para obtener óptimo, todos los puntos tiendan a seguirla.
- La **cohesión** es el comportamiento que hace que los individuos se muevan hacía un mismo punto. Actúa de forma contraria a la separación y permite concentrar los individuos hacia las posiciones más prometedoras.

Algoritmo CC-PSO

Para motivar la deducción del algoritmo de enjambre de partículas, usaremos el algoritmo CC-PSO (Centered-Centered-PSO), basándonos en los estudios del Profesor Juan Luis Fernández de la Universidad de Oviedo. El algoritmo CC-PSO, puede interpretarse físicamente como un sistema de masa-resorte amortiguado a dos muelles (Fernández - García 2008).

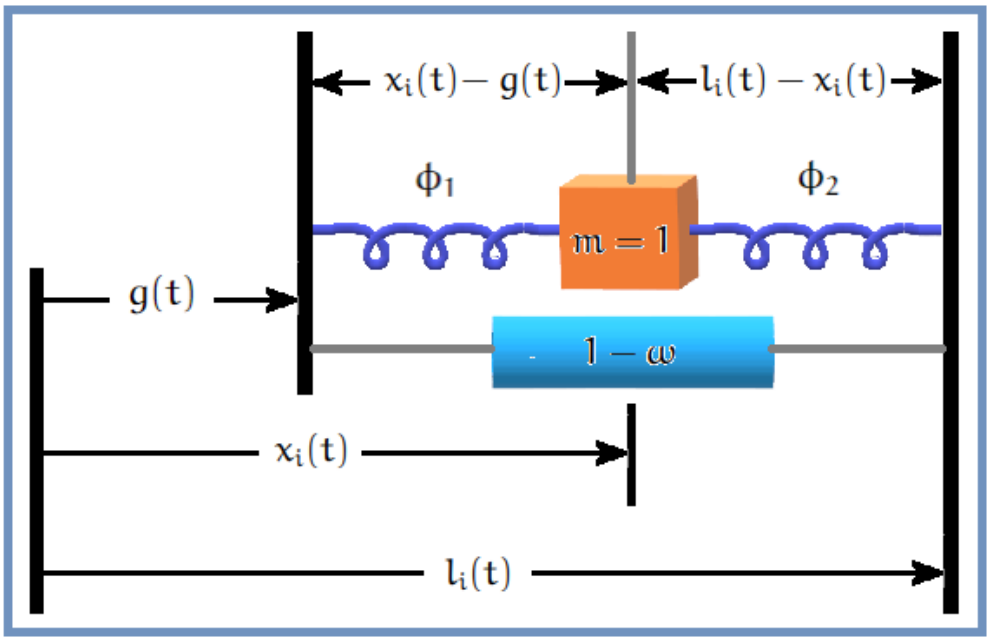


Figura 2: Sistema de masa-resorte con amortiguamiento

Considerando las contantes de restitución de cada resorte como ϕ_1 y ϕ_2. Entonces,

❶ La partícula en movimiento es un objeto de masa $m = 1$, y el amortiguamiento dado por $1 - \omega$.

❷ La posición de equilibrio para la i-ésima partícula es $l_i(t)$ y la posición global de equilibrio está dada por $g(t)$.

❸ Usando el hecho de que actúan la **Ley de Hooke** y que la fuerza de restitución dada por el amortiguamiento, es proporcional a la velocidad de la partícula, el modelo para

cada partícula del sistema está dado por:

$$\boxed{x_i''(t) + (1-\omega)x_i'(t) + (\phi_1 + \phi_2)x_i(t) = \phi_1 g(t) + \phi_2 l_i(t)}$$

En efecto, partiendo del algoritmo en su $k+1$ - ésima iteración, sabemos que:

- Cada partícula depende de su velocidad y de la aceleración local, pero además de la aceleración global, esto en el sentido del posicionamiento del líder que en este caso puede variar constantemente.
- En este sentido, para $\omega \in \mathbb{R}$ y $|\omega| < 1$ se obtiene que:

$$\boxed{\begin{cases} v_i^{k+1} = \omega v_i^k + \phi_1(g^k - x_i^k) + \phi_2(l^k - x_i^k) \\ x_i^{k+1} = x_i^k + v_i^{k+1} \end{cases}}$$

  representan las ecuaciones de actualización tanto de la velocidad como de la posición de la i - ésima partícula en la k - ésima iteración.

Tenemos entonces que:

1. ω es el peso de inercia.
2. Se definen las constantes $a_l, a_g \in \mathbb{R}$ como las **constantes de aceleración local** y **global** respectivamente.
3. Al discretizar, se obtiene un sistema de ecuaciones en diferencias estocástico.
4. En consecuencia, las constantes de aceleración local y global estocásticas verifican:

$$\boxed{\phi_1 = r_1 a_g} \qquad \text{y} \qquad \boxed{\phi_2 = r_2 a_l}$$

5. r_1 y r_2 son variables aleatorias uniformemente distribuidas en $(0,1)$ independientes para cada iteración. En consecuencia, ϕ_1, ϕ_2 son las aceleraciones aleatorias global y local.

Observe que de la segunda ecuación de actualización, se tiene que: $\boxed{v_i^{k+1} = x_i^{k+1} - x_i^k}$.

Al sustituir esta expresión en la primera ecuación, queda que:

$$x_i^{k+1} - x_i^k = \omega[x_i^k - x_i^{k-1}] + \phi_1(g^k - x_i^k) + \phi_2(l^k - x_i^k)$$

Por tanto,

$$x_i^{k+1} + (\phi_1 + \phi_2 - 1 - \omega)x_i^k + \omega x_i^{k-1} = \phi_1 g^k + \phi_2 l^k$$

lo que implica que:

$$x_i^{k+1} + (\phi - 1 - \omega)x_i^k + \omega x_i^{k-1} = \phi_1 g^k + \phi_2 l^k$$

donde $\phi = \phi_1 + \phi_2$ es una variable aleatoria con media $\overline{\phi} = \dfrac{a_l + a_g}{2}$. Considerando las condiciones iniciales de cada partícula, es decir la posición inicial y la velocidad inicial de la i - ésima partícula x_{i0}, v_{i0} y teniendo en cuenta que la posición de la i - ésima partícula en la primera iteración depende directamente de estas dos, queda que el algoritmo PSO, está dado por:

Algoritmo PSO

$$\begin{cases} x_i^{k+1} + (\phi - 1 - \omega)x_i^k + \omega x_i^{k-1} = \phi_1 g^k + \phi_2 l^k \\ x_i^0 = x_{i0} \\ x_i^1 = f(x_{i0}, v_{i0}) \end{cases}$$

para alguna relación f. Escogiendo adecuadamente los parámetros (ω, a_l, a_g) para que el punto $(\omega, \overline{\phi})$ pertenezca a la región de estabilidad de primer orden.

A saber, la región:

$$S_D = \{(\omega, \overline{\phi}) : |\omega| < 1 \ , \ 0 < \overline{\phi} < 2(1 + \omega)\}$$

determina que las trayectorias de las partículas oscilarán alrededor de sus respectivos centros de atracción, a saber

$$o_i^k = \frac{a_g g^k + a_l l^k}{a_g + a_l}$$

lo que establece directamente las condiciones de convergencia del algoritmo. Con la idea de mantener el balance entre la exploración y la explotación de los algoritmos, observamos que se pueden dar versiones del PSO con mayor tendencia a la exploración o con mayor capacidad de convergencia, que es justamente el caso del CC-PSO.

Fijando Notación

La búsqueda del óptimo por enjambre de partículas se basa en las fórmulas iterativas:

$$\begin{cases} v_{id}^{t+1} = v_{id}^{t} + c_1 R_1 (p_{id}^{t} - x_{id}^{t}) + c_2 R_2 (p_{gd}^{t} - x_{id}^{t}) & \dashrightarrow ❶ \\ x_{id}^{t+1} = x_{id}^{t} + v_{id}^{t+1} & \dashrightarrow ❷ \end{cases}$$

- v_{id}^{t} es la velocidad de una partícula dentro del enjambre y x_{id}^{t} es su posición.
- La i, es el índice de la partícula y la d indica la componente dentro del espacio de búsqueda,

Por otra parte:

- t es la época o iteración del método.
- c_1 y c_2 son parámetros de control que permiten regular la afectación de la mejor posición detectada alrededor de los distintos individuos.
- p_i es la mejor posición encontrada por la i - ésima partícula y p_g es la mejor posición encontrada por el total de las partículas.
- Los valores R_1 y R_2 son valores aleatorios generados mediante una variable aleatoria uniforme entre 0 y 1.

Los algoritmos de enjambre de partículas son especialmente adecuados para la resolución de búsqueda de óptimos en problemas de optimización de dimensión grande.

Como ya se ha mencionado su gran fortaleza se basa en reproducir la búsqueda que realizan los animales. Así pues, se trata de un método que no requiere de ningún tipo de derivadas ni otras aplicaciones que pueden no estar disponibles o no existir en el problema que se trabaja. Únicamente se requiere de la evaluación de la función.

El algoritmo de enjambre se realiza como:

1. Inicializamos n partículas del sistema. Para ello:
   - Se inicializa cada componente de la posición mediante una variable aleatoria uniforme entre el límite inferior $\mathbf{b}_l = (b_{l1}, \ldots, b_{lm})$ y el superior $\mathbf{b}_u = (b_{u1}, \ldots, b_{um})$ del espacio de búsqueda, $x_{id}^0 = U(\mathbf{b}_{ld}, \mathbf{b}_{ud})$.
   - Inicializamos la mejor posición de la partícula al mismo valor $p_i = x_i^t$.
   - Si $f(p_i) < f(p_g)$, entonces definimos $p_g = p_i$.
   - Inicializamos la velocidad de cada partícula como una variable uniforme, $v_{id} = U(-|\mathbf{b}_{ud} - \mathbf{b}_{ld}|, |\mathbf{b}_{ud} - \mathbf{b}_{ld}|)$.
2. Mientras no se cumpla el criterio de parada:
   a) Para cada partícula empleamos las ecuaciones ❶ y ❷ para actualizar su posición y velocidad.
   b) Si $f(x_i) < f(p_i)$ entonces $p_i = x_i$.
   c) Si $f(p_i) < f(p_g)$ entonces $p_g = p_i$.
3. El óptimo local encontrado viene dado por p_g.

Las condiciones de parada que se suelen incorporar son obtener un cierto valor óptimo, o que todos los iterados estén lo más cerca de una cierta tolerancia los unos de los otros.

Ejemplo 2.

Encuentra el máximo de la función:

$$f(x, y) = \text{sen}(2x) + \cos(y).$$

empleando el algoritmo de enjambre de partículas.

Para resolver este problema vamos a realizar la implementación del algoritmo en Matlab.

Primero definimos la función objetivo:

Función Objetivo

```
f=@(x,y)sin(2*x)+cos(y);
```

De esta forma podemos realizar llamadas a la función sin necesidad de crearla en un fichero adicional. A continuación inicializamos todos los parámetros del sistema:

Inicialización del Enjambre

```
%% Número de Partículas
npar=100;
%% Inicializamos particulas y velocidades
x=-pi+rand([npar,2])*2*pi;
v=(-2*pi+rand([npar,2])*4*pi)*0.1;
%% Inicializamos el óptimo por partícula
xopt=x;
%% Inicializamos el óptimo global
objfunopt=f(x(:,1),x(:,2));
[foptg,ioptg]=max(objfunopt);
xoptg=x(ioptg,:);
%% Inicializamos parámetros
c1=0.1;c2=0.2;
```

A continuación definimos el bucle principal del algoritmo de enjambre de partículas:

Bucle Principal

```
%% Inicializamos el contador de las iteraciones
it=0;
while(it<15)
    v=v+ c1 *rand(npar,2).*(xopt-x) + ...
       c2*rand(npar,2).* (xoptg.*ones(size(x))-x);
    x=x+0.2*v;
    %% Actualizamos el óptimo encontrado por ...
       cada partícula
    objfunoptn=f(x(:,1),x(:,2));
    of_mejorada=objfunoptn>objfunopt;
    xopt(of_mejorada,:)=x(of_mejorada,:);
    objfunopt(of_mejorada)=objfunoptn(of_mejorada);
    %% Actualizamos el óptimo global
    [foptg,ioptg]=max(objfunopt);
    xoptg=xopt(ioptg,:);
    %% Actualizamos el contador
    it=it+1;
end
```

Cuando ejecutamos el código anterior, obtenemos como resultado el valor $(x, y) = (-2.3562, 0.0148)$ y el valor de la función objetivo $f(x, y) = 1.9999$, que son claramente valores aproximados y que no se mantienen constantes, en función de la aleatoriedad. Además, podemos dibujar cada 5 iterados el estado de las luciérnagas tal y como se muestra en la Figura 3. Podemos ver como a medida que avanzamos, las partículas se concentran en los máximos.

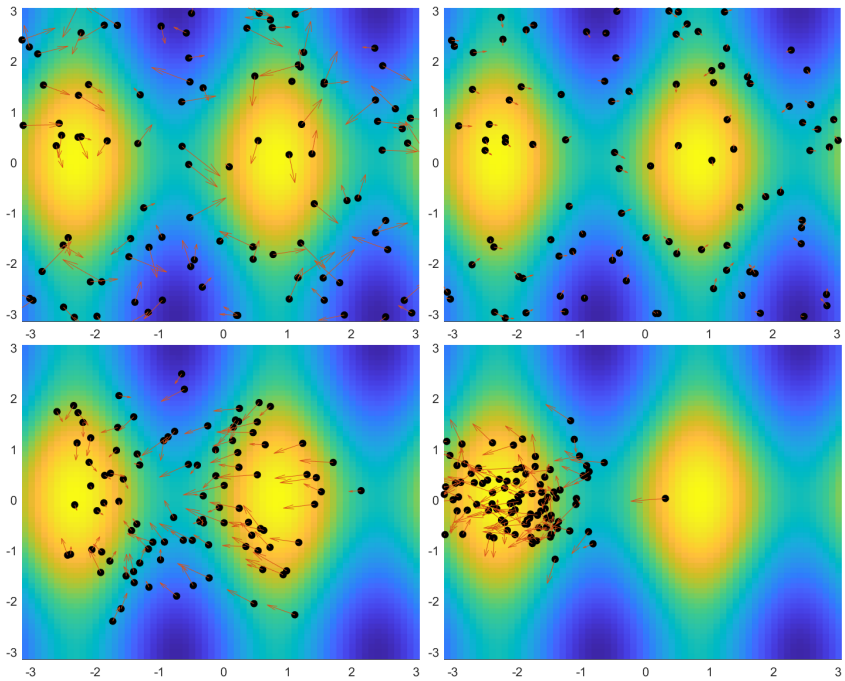


Figura 3: Representación de la función objetivo junto a las posiciones de las partículas en el estado inicial y los iterados 5, 10 y 15 (de arriba a abajo y de izquierda a derecha)

Con el paso del tiempo se han realizado varias exploraciones y seguido varias ideas para mejorar el algoritmo. La principal modificación se debe a Shi y Eberhart (Shi and Eberhart, 1998) en la que se añade un factor w a la variación de la velocidad, de forma que la fórmula ❶ resulte en:

$$v_{id}^{t+1} = wv_{id}^{t} + c_1R_1(p_{id}^{t} - x_{id}^{t}) + c_2R_2(p_{gd}^{t} - x_{id}^{t}) \dashrightarrow ❸$$

Cuando w toma un valor grande, la velocidad se incrementa dando una mayor importancia a la exploración del sistema y facilitando la búsqueda en el mismo. Mientras que si el valor es pequeño, se tiende hacia la cohesión y uniformización de todos los individuos. A menudo, llamamos el peso de inercia al parámetro w. La fórmula dada en la ecuación ❸ se considera la fórmula estándar para este tipo de problemas.

En ocasiones, las velocidades de búsqueda pueden ser muy grandes, esto provoca que las partículas escapen de la zona de búsqueda. Para evitar este comportamiento Clerc y Kennedy introdujeron un término limitante K para aumentar las probabilidades de

convergencia (Clerc and Kennedy, 2002) de forma que la Ecuación ❶ queda como:

$$v_{id}^{t+1} = K\left(v_{id}^{t} + c_1 R_1 (p_{id}^{t} - x_{id}^{t}) + c_2 R_2 (p_{gd}^{t} - x_{id}^{t})\right).$$

Como es habitual en este tipo de problemas, un factor clave consiste en fijar los parámetros y elementos del sistema. Deberemos fijar el número de partículas n que intervienen, los valores para los coeficientes c_1 y c_2 y posiblemente el valor de w o de K. Algunas de las estrategias habituales consisten en:

- Permitir variar los parámetros a lo largo del sistema. Es común emplear valores elevados de w y de K en las primeras iteraciones y reducirlos paulatinamente a medida que vamos avanzando en la iteración. De esta forma, se consigue mejorar la exploración inicial y después de las primeras iteraciones se mejora la convergencia.
- Modificar el tamaño de la población. Aunque no hay un valor óptimo y general para el valor de la población n, se ha observado que mayores valores de la población favorecen la rapidez y precisión en la convergencia del método.
- Finalmente, también se ha explorado el uso de particiones de enjambres, en las que se definen más de una familia y cada familia interactúa entre los miembros de su familia. Cada ciertos iterados se pueden modificar las familias. También se puede fijar un líder de la familia para que realice la comunicación con el resto de enjambres. Otra estrategia relacionada con esta consiste en que cada individuo se fije únicamente en los k vecinos más cercanos. De forma que favorecemos la exploración del sistema.

9.5 Algoritmos de Abejas Artificiales

Los algoritmos de abejas artificiales se basan, como su nombre indica, en la estrategia de exploración, búsqueda y recolección del nectar por parte de las abejas. Fue desarrollado por Karaboga en 2005. El algoritmo establece una analogía entre la función

objetivo y la comida o néctar, divide a los individuos en 3 tipos de abejas:

- Las **abejas empleadas** se dedican a la búsqueda de nueva comida en las zonas de búsqueda que tienen asignadas.
- Las **abejas observadoras** acuden a las zonas de búsqueda para evaluar la cantidad de comida en dicha zona.
- Las **abejas exploradoras** buscan nuevas fuentes de comida.

El algoritmo de la colonia de abejas funciona como sigue:

1. Inicialización:
   - inicializamos las fuentes de comida detectadas por las exploradoras según el valor:
   $$x_{id}^0 = b_{ld} + R(b_{ud} - b_{ld}).$$
   Donde b_l y b_u son los límites inferior y superior del espacio de búsqueda.
2. Mientras no se cumpla el criterio de parada:

   a) **Fase de las abejas empleadas**. Cada abeja empleada busca un valor que mejore el valor de la función objetivo alrededor del punto que tiene asignado. Para ello, consideramos:
   $$v_{id} = x_{id} + R_e(x_{id} - x_{kd}).$$
   Donde, R_e es un valor aleatorio en el rango $[-a, a]$ definido como parámetro de control y k es un índice que se selecciona de forma aleatoria entre las fuentes de comida. Esto genera un valor v_i. Mediante la función de *fitness*:
   $$F(x) = \begin{cases} \dfrac{1}{1+f(x)} & \text{, si } f(x) \geq 0, \\ 1 + |f(x)| & \text{, si } f(x) < m. \end{cases}$$
   Si $F(v_i) < F(x_i)$, entonces $x_i = v_i$.

b) **Fase de las abejas observadoras**. Las abejas observadoras seleccionan una fuente de alimentos de forma aleatoria de entre todas las disponibles dependiendo del valor de de la función de *fitness* asignada a la fuente. Para cada fuente se fija una probabilidad:

$$p_i = \frac{F(x_m)}{\sum_{j=1}^{n} F(x_j)}.$$

c) **Fase de las abejas exploradoras**. Después de un número de iterados N en los que una abeja no consigue mejorar el valor de la función objetivo, esta se transforma en exploradora. Una abeja exploradora seleccionará una fuente de comida dentro del espacio de búsqueda igual que se ha realizado en la fase de inicialización.

Este tipo de algoritmo suele ser sencillo de implementar y es robusto y flexible. Como en el caso anterior, no depende de las derivadas de la función objetivo, con lo que permitirá resolver ciertos tipos de problema que no podrían resolverse de otra forma. En cuanto a los parámetros del sistema, deberemos fijar la región de búsqueda, así como el rango de búsqueda alrededor de una fuente de comida α, el tamaño de la colonia de abejas y el número de iterados N que debemos aplicar para que una abeja empleada pase a ser exploradora.

9.6 Algoritmos de enjambres de luciérnagas

El último tipo de algoritmo que veremos en este tema es el **algoritmo de enjambres de luciérnagas** desarrollado por Krihnanand y Ghose en 2009. En este tipo de algoritmos los individuos que juegan un papel importante son luciérnagas. La idea fundamental será que una luciérnaga se acercará hacia aquella compañera que tenga un valor más elevado de intensidad lumínica de entre las que están a su alrededor. Para ello se definen tres parámetros para cada luciérnaga:

- **Posición** ($x_i(t)$): la posición dentro del espacio de búsqueda de nuestra función.

- **Nivel de luciferina** ($l_i(t)$): representa el nivel de brillo de la luciérnaga. Será el equivalente con memoria de la función objetivo.

- **Radio de búsqueda** ($r_i(t)$): es el radio en el que una luciérnaga es capaz de ver a sus compañeros. Si una luciérnaga está más lejos del radio de búsqueda, se considerará que no existe.

El algoritmo resultante es el siguiente:

1. **Inicialización**: Se distribuyen las luciérnagas siguiendo una distribución aleatoria, usualmente uniforme en el rango de parámetros admisibles para cada variable.

2. Mientras no se cumpla el criterio de parada:

   a) **Actualizamos el nivel de luciferina**. En esta fase se evalúa el nivel de luciferina de cada luciérnaga. Para ello, empleamos la fórmula:

   $$l_i(t) = (1-\alpha) \cdot l_i(t-1) + \gamma f(x_i(t)).$$

   Donde α es el nivel de disipación de la luciferina, γ es un parámetro de control que deberemos fijar que determina el peso de la función objetivo f.

   b) **Actualizamos la posición** de las luciernagas acorde a la función:

   $$x_i(t) = x_i(t-1) + s\left(\frac{x_k(t-1) - x_i(t-1)}{\|x_k(t-1) - x_i(t-1)\|}\right).$$

   Donde s es la longitud del paso, $\|\cdot\|$ es la norma euclídea y k es la luciérnaga hacía la que se mueve la luciérnaga m. Para seleccionar k se selecciona aleatoriamente entre aquellas vecinas en el radio de búsqueda que tienen más luciferina que la luciérnaga que estamos estudiando. Para ello, se emplean las probabilidades:

   $$p_{i,k}(t) = \frac{l_k(t) - l_i(t)}{\sum_{j \in B_i(t)} l_j - l_i}.$$

   $p_{i,k}(t)$ nos indica la probabilidad que la luciérnaga i se mueva en la dirección de la luciérnaga k y B_i es el conjunto de luciérnagas j tales que $\|x_j - x_i\| <$

$r_i(t)$ y $l_j > l_i$. Así pues, la posición de la nueva luciérnaga se moverá hacía una luciérnaga con mayor luciferina.

c) **Actualizamos el rango de búsqueda**, para ello usaremos la fórmula:

$$r_i(t+1) = \text{mín}\{r_s, \text{máx}\{0, r_i(t) + \beta(n_d - n_i(t))\}\}.$$

Donde r_s es el rango máximo para una luciérnaga, n_d es el número de vecinos deseado, $n_i(t)$ es el número de luciérnagas en el radio de observación del la i-ésima luciérnaga y β es un parámetro de control para modificar el radio de observación.

Como en el resto de casos, en este tipo de algoritmos debemos fijar un conjunto de parámetros y valores. En este caso deberemos fijar:

- El parámetro de difusión de luciferina α.
- El parámetro de peso de la función objetivo γ.
- La longitud de avance de paso de la luciérnaga s.
- El rango máximo de la luciérnaga r_s.
- El número de vecinos en el rango de observación deseado n_d.
- El parámetro de control del radio de observación β.

La elección de estos parámetros siempre dependerá de la experiencia del diseñador y del problema que estemos realizando.

Ejemplo 3.

Encontrar el máximo de la función:

$$f(x, y) = \text{sen}(2x) + \cos(y).$$

Para ello, emplea el algoritmo de enjambres de luciérnagas.

Para resolver este problema vamos a realizar la implementación del algoritmo del enjambre de luciérnagas en Matlab.

Primero definimos la función objetivo:

Función Objetivo

```
f=@(x,y)sin(2*x)+cos(y);
```

Esto nos permitirá trabajar con la función f en todo el proceso y cambiarla fácilmente en el caso de usar alguna alternativa.

A continuación inicializamos todos los parámetros del sistema:

Inicialización de Luciérnagas

```
%% Número de luciéernagas
nluc=100;
x=-pi+rand([nluc,2])*2*pi;
%% Inicializamos la luminancia de las luciernagas
l=zeros([nluc,1]);
idvec=0:nluc-1; %vector auxiliar para el cálculo
%% Parámetros del método
rs=2.5;
beta=0.1;
nd=4;
alpha=0.5;
gamma=0.7;
s=0.3;
%% Visión inicial de las luciernagas
r=ones([nluc,1])*rs/2;
```

A continuación definimos el bucle principal del algoritmo de enjambre de luciérnagas:

Bucle Principal

```
it=0;
while(it<15)
    %% Actualizamos luciferina
    l=(1-alpha)*l+gamma*f(x(:,1),x(:,2));
    %% Seleccionemos luciernaga de dirección
    dx=ones([nluc,1]).*x(:,1)'-ones([1,nluc]).*x(:,1);
    dy=ones([nluc,1]).*x(:,2)'-ones([1,nluc]).*x(:,2);
    d=dx.*dx+dy.*dy;
    ls=ones([nluc,1]).*l'-ones([1,nluc]).*l;
    ls(ls<0)=0;
    ls(d>r.*r)=0;
    probs=cumsum(ls./sum(ls,2),2);
    rr=rand([nluc,1]);
    auxra=sum(probs<rr,2);
    auxra(auxra==0)=idvec(auxra==0);
    auxra=auxra+1
    %% Actualizamos posición
    move=x(auxra,:)-x;
    moven=move./vecnorm(move,2,2);
    moven(move==0)=0;
    x=x+s*moven;
    it=it+1;
end
```

Cuando ejecutamos el código anterior, obtenemos como resultado el valor

$(x, y) = (0.7877, -0.027)$ y el valor de la función objetivo $f(x, y) = 1.9996$. Además, podemos dibujar cada 5 iterados el estado de las luciérnagas tal y como se muestra en la Figura 4. Podemos ver como a medida que avanzamos, las luciérnagas se concentran en los máximos.

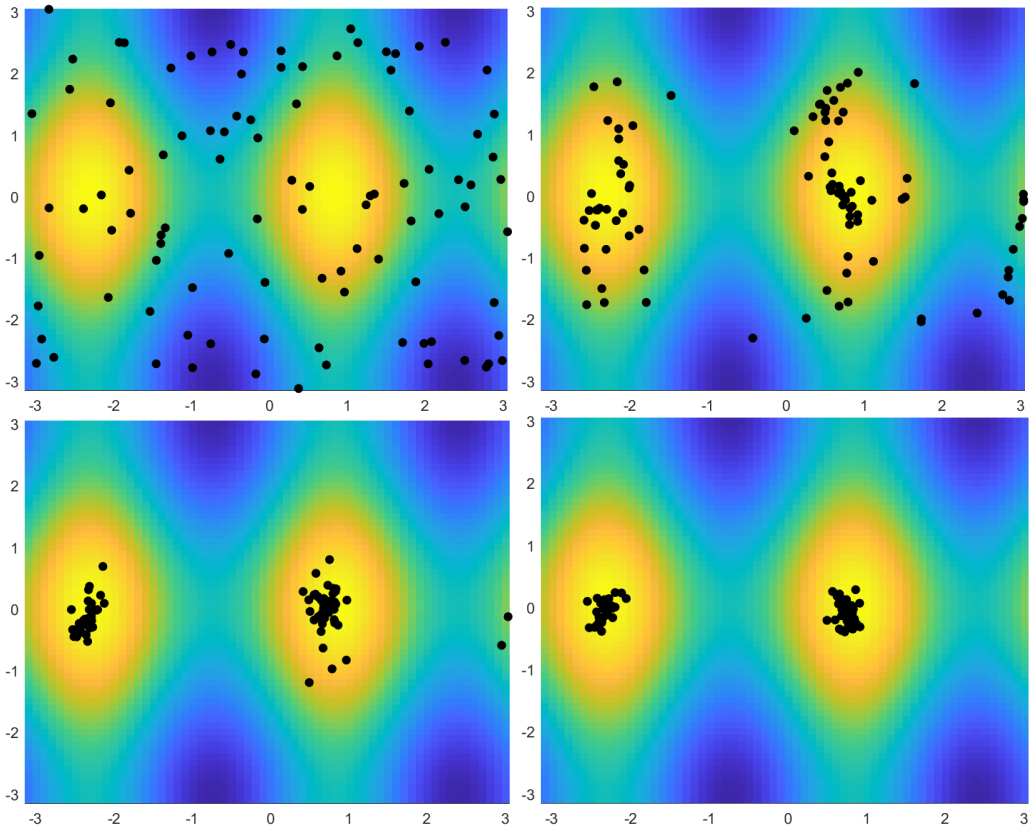


Figura 4: Representación de la función objetivo junto a las posiciones de las luciérnagas en el estado inicial y los iterados 5, 10 y 15 (de arria a abajo y de izquierda a derecha).